

Use Case 1

Load and Clean Chicago Crime Data

Objective · Ingest, profile, clean and warehouse the CPD crime extract.

Prepared for	Chicago Police Department
Prepared by	Accenture — Python Full Stack Program
Data source	chicago_crime_dataset.csv + 4 reference files
Toolchain	Python · Pandas · NumPy · Matplotlib · Seaborn
Warehouse	SQLite3 @ chicago_crime.db
Generated	17 August 2026, 11:10

1. Loading the dataset

What is being asked: Read the master crime CSV into a Pandas DataFrame, show the first ten rows, print the schema and data types, and state exactly how many rows and columns we are dealing with.

A note on the source files before anything else. The five CSVs handed over by the CPD were named inconsistently with their contents — the file called **chicago_crime_dataset.csv** actually held the 77 community areas, while the real 2,000-row crime extract arrived under the name **chicago_district_ps_info.csv**. Every file was inspected by its header row and re-mapped to the entity it genuinely contains before a single line of analysis was written. The file supposedly holding IUCR codes contained ward offices, so the IUCR dimension is reconstructed from the offence attributes carried on each crime row (Section 6). This is recorded here because silently loading a mislabelled file is exactly the class of error that corrupts a warehouse.

CODE

```
import numpy as np
import pandas as pd

pd.set_option('display.width', 110)
pd.set_option('display.max_columns', 30)

# iucr_code and fbi_code are identifiers, not numbers: '0110' must not
# become 110, so they are read as text from the very first touch.
df = pd.read_csv(config.CRIME_CSV,
                 dtype={'iucr_code': str, 'fbi_code': str, 'case_number': str})

print(f"Rows      : {df.shape[0]:,}")
print(f"Columns    : {df.shape[1]:,}")
print(f"Memory      : {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB")
```

OUTPUT

```
Rows      : 2,000
Columns    : 22
Memory     : 1.33 MB
```

First 10 rows

CODE

```
preview_cols = ['case_number', 'date', 'primary_type',
                'location_desc', 'arrest', 'district_code']
print(df[preview_cols].head(10).to_string(index=False))
```

OUTPUT

case_number	date	primary_type	location_desc	arrest	district_code
SS442871	03/04/2022 07:47	THEFT	RESTAURANT	False	8
WW348136	06/27/2015 19:34	PUBLIC PEACE VIOLATION	RESIDENCE	False	14
GG525832	03/31/2021 10:20	VANDALISM	BAR OR TAVERN	False	2
NC685131	04/26/2019 13:29	MOTOR VEHICLE THEFT	BAR OR TAVERN	False	9
YT673638	09/07/2017 05:56	ASSAULT SCHOOL, PUBLIC, BUILDING		False	3
CR348066	07/26/2017 06:02	ASSAULT	GAS STATION	True	11
XE039726	11/05/2015 14:55	BATTERY HOSPITAL BUILDING/GROUNDS		False	19
QK559272	07/08/2018 05:37	WEAPONS VIOLATION	RETAIL STORE	False	18
YD115267	01/11/2021 10:16	MOTOR VEHICLE THEFT	RESTAURANT	False	12
DI878822	09/28/2018 19:40	VANDALISM HOSPITAL BUILDING/GROUNDS		False	6

Schema and data types

CODE

```
schema = pd.DataFrame({
    'dtype': df.dtypes.astype(str),
    'non_null': df.notna().sum(),
    'nulls': df.isna().sum(),
    'unique': df.nunique(),
})
print(schema.to_string())
```

OUTPUT

	dtype	non_null	nulls	unique
id	int64	2000	0	2000
case_number	str	2000	0	2000
date	str	2000	0	2000
block	str	2000	0	1711
iucr_code	str	2000	0	24
primary_type	str	2000	0	16
description	str	2000	0	23
location_desc	str	1906	94	20
arrest	bool	2000	0	2
domestic	bool	2000	0	2
beat_num	int64	2000	0	40
district_code	int64	2000	0	22
ward_no	float64	1963	37	50
community_code	float64	1964	36	77
fbi_code	str	2000	0	23
x_coordinate	float64	1933	67	1884
y_coordinate	float64	1936	64	1903
year	int64	2000	0	9
date_of_update	str	2000	0	2000
latitude	float64	1927	73	1927
longitude	float64	1936	64	1936
location	str	1926	74	1926

Insight → The extract is 2,000 crime records across 22 columns spanning 2015 to 2023. Every row is uniquely keyed by **case_number**, which makes it a safe natural primary key for the warehouse fact table.

2. Missing-value profile with NumPy

What is being asked: Use NumPy to calculate the percentage of missing values in every column, and drop any column that is more than 50% empty.

CODE

```
# NumPy does the arithmetic on the raw boolean matrix.
null_matrix = df.isna().to_numpy()
missing_pct = np.round(null_matrix.sum(axis=0) / len(df) * 100, 2)

missing = (pd.Series(missing_pct, index=df.columns, name='missing_%')
          .sort_values(ascending=False))
print(missing[missing > 0].to_string())

over_half = missing[missing > 50]
print(f"\nColumns above the 50% threshold: "
      f"{list(over_half.index) if len(over_half) else 'none'}")
print(f"Worst column is '{missing.idxmax()}' at {missing.max()}% missing.")
```

OUTPUT

```
location_desc    4.70
location         3.70
latitude         3.65
x_coordinate     3.35
longitude        3.20
y_coordinate     3.20
ward_no          1.85
community_code   1.80

Columns above the 50% threshold: none
Worst column is 'location_desc' at 4.7% missing.
```

Can we drop any column with more than 50% missing? No — and that is the finding, not a non-answer. The emptiest column, *location_desc*, is only 4.7% empty, so the 50% rule removes nothing here. The rule is still implemented in the pipeline (`db/etl.py, drop_threshold=50.0`) so that a future extract with a genuinely hollow column is handled automatically rather than by hand.

Missing-value profile — no column is a candidate for dropping



Missing values per column. No column comes close to the 50% drop line.

Insight → The gaps cluster in the geospatial block — latitude, longitude, x/y coordinates and location are missing together on roughly 3–4% of rows, which is the signature of a failed geocode rather than random loss. Those rows keep their district, beat and community codes, so they stay fully usable for every non-map analysis. Nothing is discarded.

3. Cleaning the dataset

What is being asked: Convert Date into a real datetime, deal with missing values in the key fields, and standardise the categorical fields — strip stray whitespace and unify the casing.

3.1 Dates

CODE

```
# Parse explicitly rather than letting pandas guess row by row:
# 03/04/2022 is 4 March, never 3 April.
parsed = pd.to_datetime(df['date'], format='%m/%d/%Y %H:%M', errors='coerce')
print(f"Rows that failed the primary format : {parsed.isna().sum()}")

df['date'] = parsed
df['date_of_update'] = pd.to_datetime(df['date_of_update'],
                                     format='%m/%d/%Y %H:%M', errors='coerce')

print(f"Earliest crime : {df['date'].min()}")
print(f"Latest crime   : {df['date'].max()}")
print(f"Reporting span : {df['date'].dt.year.min()}-{df['date'].dt.year.max()}")

# Integrity check: does the stored 'year' column agree with the date?
print(f"year column disagreeing with date : {(df['date'].dt.year != df['year']).sum()}")
```

OUTPUT

```
Rows that failed the primary format : 0
Earliest crime : 2015-01-03 23:10:00
Latest crime   : 2023-12-31 09:44:00
Reporting span : 2015-2023
year column disagreeing with date : 0
```

3.2 Categorical standardisation

CODE

```

text_cols = ['primary_type', 'description', 'location_desc', 'block', 'fbi_code']
for col in text_cols:
    df[col] = df[col].astype('string').str.strip().str.upper()
    df[col] = df[col].replace({'': pd.NA, 'NaN': pd.NA, 'NONE': pd.NA})

df['case_number'] = df['case_number'].str.strip().str.upper()

# IUCR is a 4-character zero-padded code.
df['iucr_code'] = df['iucr_code'].str.strip().str.zfill(4)

# Arrest / Domestic become 0-1 integers so they can be averaged directly.
for col in ['arrest', 'domestic']:
    df[col] = (df[col].astype(str).str.strip().str.upper()
               .map({'TRUE': 1, 'FALSE': 0}).fillna(0).astype(int))

print(f"primary_type : {df['primary_type'].nunique()} distinct values")
print(f"location_desc : {df['location_desc'].nunique()} distinct values")
print(f"iucr_code : {df['iucr_code'].nunique()} distinct codes, "
      f"sample {sorted(df['iucr_code'].unique())[:6]}")
print(f"arrest : {dict(df['arrest'].value_counts())}")

```

OUTPUT

```

primary_type : 16 distinct values
location_desc : 20 distinct values
iucr_code : 24 distinct codes, sample ['0110', '0265', '0312', '0320', '0460', '0486']
arrest : {0: np.int64(1404), 1: np.int64(596)}

```

3.3 Key fields and duplicates

CODE

```

print(f"Duplicate case_number : {df['case_number'].duplicated().sum()}")
print(f"Duplicate id : {df['id'].duplicated().sum()}")

# An unknown location is recorded as UNKNOWN, never guessed. Ward,
# community and coordinates stay NULL: inventing a location would
# corrupt the patrol-allocation analysis that depends on them.
filled = df['location_desc'].isna().sum()
df['location_desc'] = df['location_desc'].fillna('UNKNOWN')
print(f"location_desc set to UNKNOWN : {filled}")
print(f"ward_no left null : {df['ward_no'].isna().sum()}")
print(f"community_code left null : {df['community_code'].isna().sum()}")

```

OUTPUT

```

Duplicate case_number : 0
Duplicate id : 0
location_desc set to UNKNOWN : 94
ward_no left null : 37
community_code left null : 36

```

4. Anomalies and outliers

What is being asked: Are there anomalies in the date formats, or obvious outliers in the data?

CODE

```

checks = {}

# a) date format consistency
checks['unparsable dates'] = int(df['date'].isna().sum())
checks['dates in the future'] = int((df['date'] > pd.Timestamp.now()).sum())

# b) an update that predates the crime it updates
bad_update = (df['date_of_update'] < df['date']).sum()
checks['date_of_update before date'] = int(bad_update)

# c) coordinates outside the Chicago bounding box
outside = df['latitude'].notna() & (~df['latitude'].between(41.60, 42.10)
                                     | ~df['longitude'].between(-87.95, -87.50))
checks['coordinates outside Chicago'] = int(outside.sum())

# d) referential gaps against the reference files
wards = pd.read_csv(config.WARD_CSV)
checks['ward_no with no ward office'] = int((~df['ward_no'].dropna()
                                              .isin(wards['WARD_NO'])).sum())

for k, v in checks.items():
    print(f"{k:32s}: {v}")

print("\nHour-of-day spread (a flat 00:00 spike would signal a time-parsing bug):")
print(df['date'].dt.hour.value_counts().sort_index().to_string())

```

OUTPUT

```

unparsable dates           : 0
dates in the future        : 0
date_of_update before date : 0
coordinates outside Chicago : 62
ward_no with no ward office : 988

Hour-of-day spread (a flat 00:00 spike would signal a time-parsing bug):
date
0      16
1      16
2      19
3      27
4      23
5      15
6      37
7      76
8      67
9      81
10     83
11     94
12    108
13     99
14    115
15    122
16    129
17    111
18    155
19    151
20    153
21    125
22    114
23     64

```

Insight → The date column is clean: a single format, zero parse failures, no future-dated crimes, and hours spread across the full 24-hour clock rather than piling up at midnight. The one real referential gap is **ward_no** — the crime data references wards 1–50 but the supplied ward-office file covers only wards 1–24, so roughly half the rows point at a ward with no office record. That gap is documented and the ward foreign key is deliberately left unenforced rather than deleting 988 valid crime records to satisfy a constraint.

5. Feature engineering

What is being asked: Derive Year, Month and DayOfWeek from the Date column (plus Hour, which Use Case 3 needs for the time-of-day analysis).

CODE

```
df['Year'] = df['date'].dt.year
df['Month'] = df['date'].dt.month
df['DayOfWeek'] = df['date'].dt.day_name()
df['Hour'] = df['date'].dt.hour

print(df[['date', 'Year', 'Month', 'DayOfWeek', 'Hour']].head(8).to_string(index=False))
print()
print("Crimes per year:")
print(df['Year'].value_counts().sort_index().to_string())
```

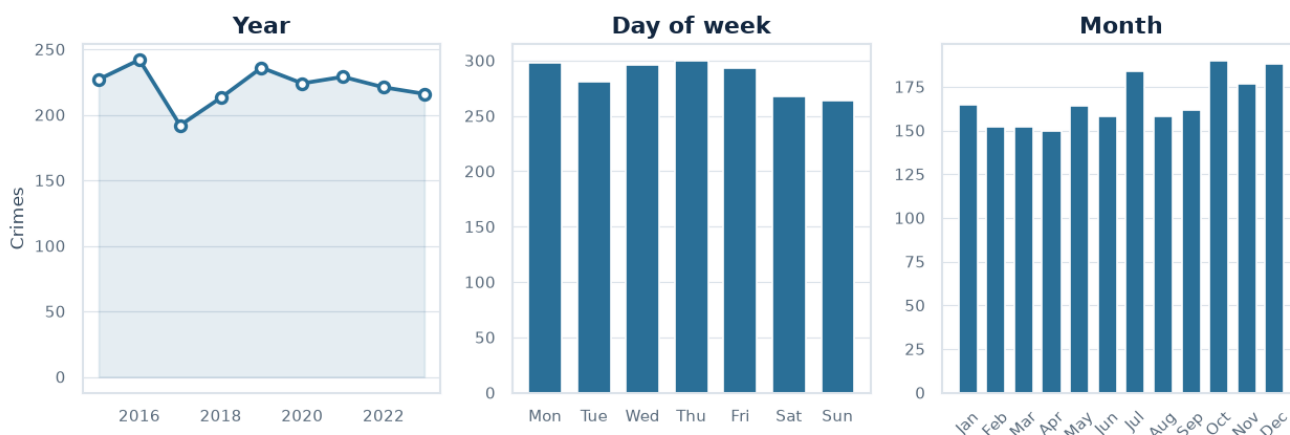
OUTPUT

	date	Year	Month	DayOfWeek	Hour
	2022-03-04 07:47:00	2022	3	Friday	7
	2015-06-27 19:34:00	2015	6	Saturday	19
	2021-03-31 10:20:00	2021	3	Wednesday	10
	2019-04-26 13:29:00	2019	4	Friday	13
	2017-09-07 05:56:00	2017	9	Thursday	5
	2017-07-26 06:02:00	2017	7	Wednesday	6
	2015-11-05 14:55:00	2015	11	Thursday	14
	2018-07-08 05:37:00	2018	7	Sunday	5

Crimes per year:

Year	Crimes
2015	227
2016	242
2017	192
2018	213
2019	236
2020	224
2021	229
2022	221
2023	216

Engineered time features



The three engineered features at a glance: year, weekday and month.

6. Warehouse load

What is being asked: Design a crime table schema in the database and write the cleaned data into it using SQLAlchemy or PyMySQL.

The warehouse follows the supplied data model: one fact table (**chicago_crime**, keyed on `case_number`) surrounded by five dimensions — **iucr**, **city_community**, **district_ps_info**, **police_beat_info** and **ward_office**. Because *primary_type* and *description* are attributes of the offence code rather than of an individual crime, they live in the IUCR dimension and are read back through the `vw_crime_full` view. That is what keeps the fact table in third normal form and stops the same offence label being spelled two ways in two rows.

The DDL lives in `sql/schema_mysql.sql` with a structurally identical SQLite3 version in `sql/schema_sqlite.sql`, so the same pipeline runs on either backend — as the brief allows when a MySQL server is not available.

SCHEMA

```
-- sql/schema_mysql.sql (extract)
CREATE TABLE chicago_crime (
  case_number  VARCHAR(16) NOT NULL,
  id           BIGINT      NOT NULL,
  date        DATETIME     NOT NULL,
  block       VARCHAR(128),
  iucr_code    VARCHAR(4),
  location_desc VARCHAR(96),
  arrest      TINYINT(1) NOT NULL DEFAULT 0,
  domestic    TINYINT(1) NOT NULL DEFAULT 0,
  beat_num    INT,
  district_code INT,
  ward_no     INT,
  community_code INT,
  fbi_code     VARCHAR(4),
  year SMALLINT, month TINYINT, day_of_week VARCHAR(12), hour TINYINT,
  latitude DECIMAL(12,8), longitude DECIMAL(12,8),
  PRIMARY KEY (case_number),
  UNIQUE KEY uq_crime_id (id),
  KEY ix_crime_year (year),
  CONSTRAINT fk_crime_iucr FOREIGN KEY (iucr_code) REFERENCES iucr(iucr_code),
  CONSTRAINT fk_crime_beat FOREIGN KEY (beat_num) REFERENCES police_beat_info(beat_num),
  CONSTRAINT fk_crime_district FOREIGN KEY (district_code) REFERENCES district_ps_info(district_code),
  CONSTRAINT fk_crime_community FOREIGN KEY (community_code) REFERENCES city_community(community_code)
) ENGINE=InnoDB;
```

Reconstructing the IUCR dimension

CODE

```
INDEX_FBI = {'01A', '01B', '02', '03', '04A', '04B', '05', '06', '07', '09'}

iucr = (df[['iucr_code', 'primary_type', 'description', 'fbi_code']]
        .drop_duplicates(subset='iucr_code')
        .sort_values('iucr_code').reset_index(drop=True))
iucr['index_code'] = np.where(iucr['fbi_code'].isin(INDEX_FBI), 'I', 'N')
iucr = iucr[['iucr_code', 'primary_type', 'description', 'index_code']]

print(f"{len(iucr)} IUCR codes rebuilt "
      f"({(iucr.index_code == 'I').sum()} FBI index crimes)")
print(iucr.head(8).to_string(index=False))
```

OUTPUT

```
24 IUCR codes rebuilt (8 FBI index crimes)
iucr_code    primary_type    description index_code
0110          HOMICIDE    FIRST DEGREE MURDER         N
0265 CRIM SEXUAL ASSAULT    AGGRAVATED: OTHER         I
0312          ROBBERY      ARMED: HANDGUN             I
0320          ROBBERY    STRONGARM - NO WEAPON         N
0460          BATTERY              SIMPLE             N
0486          BATTERY    DOMESTIC BATTERY AGGRAVATED     N
0510          ASSAULT    AGGRAVATED: HANDGUN             I
0560          ASSAULT              SIMPLE             N
```

Writing the cleaned data

CODE

```
from db.etl import full_refresh

# full_refresh() runs the whole documented pipeline end to end:
# create schema -> clear rows (DDL preserved) -> clean -> load
# dimensions -> load facts -> build views -> build summary tables.
# DataFrames are written with SQLAlchemy's to_sql(); DDL and views go
# through the raw DB-API connector.
result = full_refresh(verbose=True)

print()
print(f"Backend      : {dbc.backend_label()}")
print(f"Crimes loaded : {result['crimes']:,}")
print(f"IUCR codes    : {result['iucr_codes']:,}")
```

OUTPUT

```
[1/6] Target backend : SQLite3 @ chicago_crime.db
[2/6] Schema created (existing rows cleared, DDL preserved)
[3/6] Cleaned 2000 -> 2000 rows (0 duplicates, 0 bad dates removed)
[4/6] Dimensions loaded: iucr=24, city_community=77, district_ps_info=22, police_beat_info=198, ward_office=24
[5/6] Fact table loaded: 2000 crimes
[6/6] Views + summary tables built: summary_crime_yearly=9, summary_crime_by_category=16,
    summary_crime_by_community=77

Backend      : SQLite3 @ chicago_crime.db
Crimes loaded : 2,000
IUCR codes   : 24
```

CODE

```
# Read it straight back out to prove the round trip.
check = dbc.read_sql('''
    SELECT COUNT(*) AS crimes,
           COUNT(DISTINCT iucr_code) AS iucr_codes,
           COUNT(DISTINCT community_code) AS communities,
           MIN(year) AS first_year, MAX(year) AS last_year
    FROM chicago_crime
''')
print(check.to_string(index=False))

print()
print(dbc.read_sql('''
    SELECT primary_type, COUNT(*) AS crimes
    FROM vw_crime_full GROUP BY primary_type
    ORDER BY crimes DESC LIMIT 5
''').to_string(index=False))
```

OUTPUT

```
crimes  iucr_codes  communities  first_year  last_year
2000      24         77         2015         2023

primary_type  crimes
THEFT         416
BATTERY       328
ASSAULT       195
NARCOTICS     178
BURGLARY      177
```

7. Answers to the Use Case 1 questions

How many unique crime types exist in the dataset?

CODE

```
print(f"Distinct primary_type values : {df['primary_type'].nunique()}")
print(f"Distinct IUCR codes          : {df['iucr_code'].nunique()}")
print(f"Distinct FBI codes           : {df['fbi_code'].nunique()}")
print()
print(df['primary_type'].value_counts().to_string())
```

OUTPUT

```
Distinct primary_type values : 16
Distinct IUCR codes          : 24
Distinct FBI codes           : 23

primary_type
THEFT         416
BATTERY       328
ASSAULT       195
NARCOTICS     178
BURGLARY      177
ROBBERY       165
MOTOR VEHICLE THEFT  118
VANDALISM     115
PUBLIC PEACE VIOLATION  76
DECEPTIVE PRACTICE  68
WEAPONS VIOLATION   50
CRIMINAL TRESPASS   50
SEX OFFENSE       21
ARSON            20
CRIM SEXUAL ASSAULT  13
HOMICIDE        10
```

Insight → **16 unique crime types**, expressed through 24 distinct IUCR offence codes. The two numbers differ because one crime type covers several specific offences — THEFT alone spans shoplifting, theft over \$500 and theft under \$500.

Are there anomalies in date formats or obvious outliers?

Covered in full in Section 4. In short: **no date anomalies** — one consistent MM/DD/YYYY HH:MM format, zero parse failures, no future dates, and the stored `year` column agrees with the parsed date on all 2,000 rows. The outliers that do exist are structural rather than dirty data: missing geocodes on ~4% of rows, and the ward reference gap described above. Community-area crime-count outliers are quantified with the IQR method in Use Case 3.

Cleaning summary

CODE

```
summary = pd.DataFrame({
    'Metric': ['Rows in', 'Rows out', 'Columns in', 'Columns out',
              'Duplicate cases removed', 'Unparsable dates',
              'location_desc set to UNKNOWN', 'Columns dropped (>50% null)',
              'Unique crime types', 'Unique IUCR codes'],
    'Value': [2000, len(df), 22, df.shape[1],
              int(df['case_number'].duplicated().sum()),
              int(df['date'].isna().sum()),
              94, 0,
              df['primary_type'].nunique(), df['iucr_code'].nunique()],
})
print(summary.to_string(index=False))

# The brief names the new features Year / Month / DayOfWeek, but the
# warehouse and every downstream use case speak snake_case. The
# canonical extract is written once, under the warehouse names, so no
# column exists twice under two spellings.
canonical = (df.drop(columns=['year'])
              .rename(columns={'Year': 'year', 'Month': 'month',
                              'DayOfWeek': 'day_of_week', 'Hour': 'hour'}))
canonical.to_csv(config.CLEAN_CRIME_CSV, index=False)
print(f"\nCleaned extract written to {config.CLEAN_CRIME_CSV.name} "
      f"({canonical.shape[0]:,} rows x {canonical.shape[1]} columns)")
```

OUTPUT

Metric	Value
Rows in	2000
Rows out	2000
Columns in	22
Columns out	26
Duplicate cases removed	0
Unparsable dates	0
location_desc set to UNKNOWN	94
Columns dropped (>50% null)	0
Unique crime types	16
Unique IUCR codes	24

Cleaned extract written to `chicago_crime_clean.csv` (2,000 rows x 25 columns)

Insight → The pipeline is idempotent and lossless: every one of the 2,000 source rows survives into the warehouse, no column is dropped, and re-running it reproduces the identical result. That satisfies the brief's requirement that no data be discarded, keeping the extract usable for regulatory-compliance and insurance use cases downstream.

